Funcionalidades úteis do Django Shell

O shell do Django permite que você escreva instruções Python a partir da linha de comando como se elas estivessem sendo executadas dentro do Django Web Framework.

O shell do Django é muito poderoso, mas muitos desenvolvedores iniciantes subestimam sua utilidade.

Este tutorial descreve alguns recursos úteis do shell do Django.

Índice

Você pode pular para uma seção específica deste tutorial do Django Shell usando o índice abaixo:

- O que é o Django Shell?
  - Acessando o Django Shell
- Inserindo dados no banco de dados
  - Inserindo vários registros
- Pesquisando no banco de dados
  - Procurando por todos os objetos
  - Busca por um único registro
  - Recuperar dados específicos de um registro
- Pesquisas de campo
  - contém operador
  - Operadoras GTE e LTE
- Atualização de registros
  - Atualizando um único valor
  - Atualizando múltiplos valores
  - Atualizando vários registros
- Excluindo um registro
  - Excluindo um único registro
  - Excluindo vários registros
- Considerações finais

O que é o Django Shell?

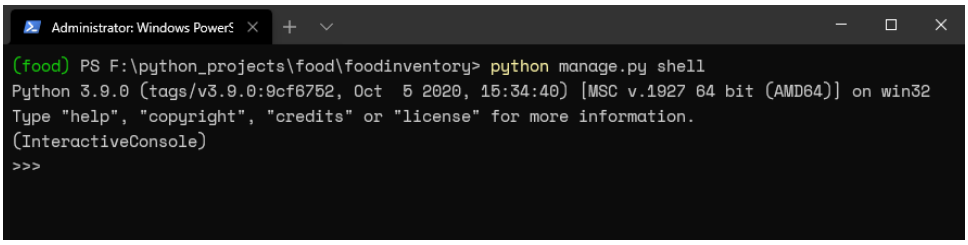
O shell do Django é um ambiente de linha de comando interativo que combina a funcionalidade do framework Django com o shell padrão do Python. O shell do Django carrega os parâmetros e configurações específicos do projeto, que funcionam apenas dentro desse projeto, permitindo ao usuário isolar o ambiente de trabalho e se concentrar naquele projeto em particular.

Uma das principais funcionalidades que este shell oferece é o fácil acesso ao mapeador objeto-relacional (ORM), que permite ao usuário interagir diretamente com o banco de dados. O ORM é responsável por executar consultas ao banco de dados dentro de um projeto Django. O mapeador objeto-relacional reduz a necessidade de conhecimento aprofundado de bancos de dados relacionais e elimina a necessidade de usar consultas SQL no projeto na maioria dos casos.

Acessando o Django Shell

O shell do Django pode ser acessado usando o `shell`comando em um projeto Django. Ao longo deste artigo, usaremos um projeto Django padrão chamado "foodinventory" dentro do ambiente virtual Python "food".

```
python manage.py shell
```



Antes de nos aprofundarmos no shell do Django, criaremos um aplicativo Django chamado "fruits" dentro do projeto "foodinventory". Em seguida, popularemos o arquivo models.py criando uma tabela "FruitsInfo" e migraremos as alterações para criar um banco de dados SQLite simples.

```
python manage.py startapp fruits
```

modelos.py

```
from django.db import models

class FruitsInfo(models.Model):
    name = models.CharField(max_length=30)
    origin = models.CharField(max_length=60)
    price = models.DecimalField(max_digits=4,null=False,decimal_places
    availability = models.IntegerField(default=0)

    def __str__(self):
            return self.origin + " " + self.name
```

.\foodinventory\settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'fruits'
]
```

```
python manage.py makemigrations
```

RESULTADO

```
(food) PS F:\python_projects\food\foodinventory>
python manage.py makemigrations
Migrations for 'fruits':
  fruits\migrations\0001_initial.py
    - Create model FruitsInfo
```

```
python manage.py migrate
```

RESULTADO

```
(food) PS F:\python_projects\food\foodinventory> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, fruits, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying fruits.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Inserindo dados no banco de dados

Em Django, uma classe de modelo representa uma tabela de banco de dados e uma instância dessa classe representa um registro específico dentro do banco de dados. Isso é análogo ao uso de uma instrução INSERT em SQL. Um registro pode ser criado simplesmente instanciando a classe de modelo usando os argumentos de palavra-chave definidos e chamando o método save() para confirmar o novo registro no banco de dados.

No exemplo a seguir, veremos como adicionar um novo registro à classe de modelo FruitsInfo. Primeiro, precisamos importar a classe “FruitsInfo” para o Django Shell, depois criar um objeto da classe “FruitsInfo” chamado record usando todos os argumentos nomeados e, finalmente, usar o método save() para inserir o registro no banco de dados.

```
from fruits.models import FruitsInfo
record = FruitsInfo(name="apple",origin="USA",price=2.5,availability=1
record.save()
```

RESULTADO

```
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct  5 2020, 15:34:40) [MSC v.1927 64 bit (AMD
64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from fruits.models import FruitsInfo
>>> record = FruitsInfo(name="apple",origin="USA",price=2.5,availability=1000)
>>> record.save()
>>>
```

Como não há erros indicados no shell do Django, podemos assumir que o registro foi adicionado com sucesso ao banco de dados. Vamos verificar isso usando o método “all()” para buscar todos os registros do banco de dados.

```
FruitsInfo.objects.all()
```

RESULTADO

```
>>> FruitsInfo.objects.all()
<QuerySet [<FruitsInfo: USA apple>]>
>>>
```

Como definimos um método “__str__()” para exibir um objeto em um formato legível para humanos, o método “all()” exibirá apenas o valor definido no método “__str__()”. O método value() permite que os usuários extraiam os valores de um determinado objeto, conforme mostrado abaixo.

```
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'USA', 'price': Decimal('2.50'),
'availability': 1000}]>
>>>
```

Outra forma de inserir um registro em uma classe de modelo é usar o método “create()”. Isso elimina a necessidade de chamar o método save() para confirmar o registro no banco de dados.

```
from fruits.models import FruitsInfo
record = FruitsInfo.objects.create(name="orange",origin="canada",price
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> record = FruitsInfo.objects.create(name="orange",origin="canada",price=1.25,a
vailability=2000)
>>>
```

Vamos verificar se o registro foi inserido no banco de dados usando o método “all()”.

```
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> FruitsInfo.objects.all()
<QuerySet [<FruitsInfo: USA apple>, <FruitsInfo: canada orange>]>
>>>
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'USA', 'price': Decimal('2.50'),
'availability': 1000}, {'id': 2, 'name': 'orange', 'origin': 'canada', 'price': D
ecimal('1.25'), 'availability': 2000}]>
>>>
```

O resultado acima mostra que o novo registro foi criado sem a necessidade de declarar o método “save()”.

Inserindo vários registros

Nesta seção, veremos como inserir vários registros em uma classe específica. Criaremos uma nova classe `FruitsVendors` dentro do arquivo `models.py` no aplicativo de frutas e migraremos as alterações.

Arquivo models.py modificado

```
from django.db import models

class FruitsInfo(models.Model):
    name = models.CharField(max_length=30)
    origin = models.CharField(max_length=60)
    price = models.DecimalField(max_digits=4,null=False,decimal_places
    availability = models.IntegerField(default=0)

    def __str__(self):
            return self.origin + " " + self.name


class FruitsVendors(models.Model):
    vendor_id = models.CharField(max_length=4, null=False, primary_key
    vendor_name = models.CharField(max_length=60)
    vendor_location = models.CharField(max_length=60)

    def __str__(self):
        return self.vendor_id + " - " + self.vendor_name + " - " + sel
```

Na nova classe FruitsVendors, definimos um campo de chave primária chamado “vendor_id”. Em seguida, definimos o “ **str** ()” para exibir todos os dados dentro da classe em uma string formatada.

Mudanças na Migração

```
python manage.py makemigrations
python manage.py migrate
```

RESULTADO

```
(food) PS F:\python_projects\food\foodinventory> python manage.py makemigrations
Migrations for 'fruits':
  fruits\migrations\0002_fruitsvendors.py
    - Create model FruitsVendors
(food) PS F:\python_projects\food\foodinventory> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, fruits, sessions
Running migrations:
  Applying fruits.0002_fruitsvendors... OK
```

Vamos ver como inserir vários registros na classe FruitsVendors de uma só vez usando o método bulk_create() e o shell do Django.

```
from fruits.models import FruitsVendors

FruitsVendors.objects.bulk_create([FruitsVendors(vendor_id="V001", ven
```

RESULTADO

```
>>> from fruits.models import FruitsVendors
>>> FruitsVendors.objects.bulk_create([
... FruitsVendors(vendor_id="V001", vendor_name="Fresh Fruits", vendor_location = "New York"),
... FruitsVendors(vendor_id="V002", vendor_name="Direct Delivery", vendor_location = "Sao Paulo"),
... FruitsVendors(vendor_id="V003", vendor_name="Fruit Mate", vendor_location = "Sydney"),
... ])
[<FruitsVendors: V001 - Fresh Fruits - New York>, <FruitsVendors: V002 - Direct Delivery - Sao Paulo>,
 <FruitsVendors: V003 - Fruit Mate - Sydney>]
>>>
```

Pesquisando no banco de dados

Com o shell do Django, podemos pesquisar objetos em um banco de dados. Ao pesquisar, podemos usar filtros para extrair objetos específicos. Isso é análogo às instruções SELECT e WHERE em SQL. Vejamos como pesquisar registros na classe FruitsInfo.

Procurando por todos os objetos

Usando o método `all()`, podemos obter dados de todos os registros em uma tabela. Se o método ` **str** ()` for declarado na classe, ele será a saída do método `all()`. Para obter os valores dentro do objeto, podemos adicionar o método `values()` ao método `all()`.

```
from fruits.models import FruitsInfo
FruitsInfo.objects.all()
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> FruitsInfo.objects.all()
<QuerySet [<FruitsInfo: USA apple>, <FruitsInfo: canada orange>, <FruitsInfo: bra
zil pineapple>, <FruitsInfo: china kiwi>]>
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'USA', 'price': Decimal('2.50'),
'availability': 1000}, {'id': 2, 'name': 'orange', 'origin': 'canada', 'price': D
ecimal('1.25'), 'availability': 2000}, {'id': 3, 'name': 'pineapple', 'origin': '
brazil', 'price': Decimal('3.00'), 'availability': 1500}, {'id': 4, 'name': 'kiwi
', 'origin': 'china', 'price': Decimal('1.00'), 'availability': 3000}]>
>>>
```

Busca por um único registro

O método `all()` com o filtro pode ser usado para recuperar um único registro de uma tabela. Isso é feito usando o método `filter()`. Os exemplos a seguir demonstram como pesquisar registros pelos campos `id` e `name`.

Pesquisar registros pelo campo inteiro “id”

```
from fruits.models import FruitsInfo
FruitsInfo.objects.all().filter(id=1)
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> FruitsInfo.objects.all().filter(id=1)
<QuerySet [<FruitsInfo: USA apple>]>
>>>
```

```
FruitsInfo.objects.all().values().filter(id=1)
```

RESULTADO

```
>>> FruitsInfo.objects.all().values().filter(id=1)
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'USA', 'price': Decimal('2.50'),
 'availability': 1000}]>
>>>
```

Pesquisar registros pelo campo de texto “nome”

```
FruitsInfo.objects.all().filter(name='orange')

FruitsInfo.objects.all().values().filter(name='orange')
```

RESULTADOS

```
>>> FruitsInfo.objects.all().filter(name='orange')
<QuerySet [<FruitsInfo: canada orange>]>
>>>
>>> FruitsInfo.objects.all().values().filter(name='orange')
<QuerySet [{'id': 2, 'name': 'orange', 'origin': 'canada', 'price': Decimal('1.2
5'), 'availability': 2000}]>
>>>
```

O método `filter()` sempre retornará um `QuerySet` do Django, mesmo que haja apenas um registro na saída. Se quisermos recuperar um único registro, podemos usar o método `get()`. No entanto, se houver mais de um registro que corresponda à consulta especificada no método `get()`, isso resultará em um erro "MultipleObjectsReturned".

O método get() é mais viável quando buscamos usando campos de índice únicos, como a chave primária. Vejamos como o método get() pode ser utilizado com o campo "id".

```
record = FruitsInfo.objects.get(id=3)
print(record)
```

RESULTADO

```
>>> record = FruitsInfo.objects.get(id=3)
>>> print(record)
brazil pineapple
>>>
```

Recuperar dados específicos de um registro

Ao usar o método get() para recuperar um registro específico, podemos extrair as informações de um campo definido usando a nova instância criada durante a busca pelo registro. No exemplo a seguir, criamos uma nova instância chamada “record” para a classe FruitsInfo, onde o campo “id” é igual a “3”. Em seguida, extraímos as informações de cada campo.

```
record = FruitsInfo.objects.get(id=3)
print(record)

print(record.name)
print(record.origin)
print(record.price)
print(record.availability)
```

RESULTADOS

```
>>> record = FruitsInfo.objects.get(id=3)
>>> print(record)
brazil pineapple
>>>
>>> print(record.name)
pineapple
>>> print(record.origin)
brazil
>>> print(record.price)
3.00
>>> print(record.availability)
1500
>>>
```

Vamos criar um demonstrativo de impressão personalizado usando cada campo do registro.

```
print(f"The stock of {record.name} which originates from {record.origi
```

RESULTADO

```
>>> print(f"The stock of {record.name} which originates from {record.origin.titl
e()} is at {record.availability} and each {record.name} is priced at {record.pri
ce}.")
The stock of pineapple which originates from Brazil is at 1500 and each pineappl
e is priced at 3.00.
>>>
```

Pesquisas de campo

No mapeador objeto-relacional do Django, podemos especificar operadores para filtrar ainda mais o conjunto de dados resultante. Isso é análogo aos operadores que podem ser especificados em uma instrução WHERE do SQL. Alguns exemplos de pesquisas de campos e seus respectivos operadores SQL são:

- contém - CURTIR
- intervalo - ENTRE
- gte (maior ou igual a) →=
- lte (menor ou igual a) - <=

Os exemplos a seguir demonstram como podemos usar as pesquisas de campos no Django Shell.

contém operador

Vamos procurar por nomes de fornecedores que incluam o nome "Fruit" na classe FruitsVendors.

```
from fruits.models import FruitsVendors
FruitsVendors.objects.filter(vendor_name__contains='Fruit')
```

RESULTADO

```
>>> from fruits.models import FruitsVendors
>>> FruitsVendors.objects.filter(vendor_name__contains='Fruit')
<QuerySet [<FruitsVendors: V001 - Fresh Fruits - New York>, <FruitsVendors: V003 - Fruit Mate - Sydney>]>
>>>
```

Operadoras GTE e LTE

Nos exemplos a seguir, procuraremos registros em que o preço seja maior ou igual a 2 e a disponibilidade seja menor ou igual a 1000 na classe FruitsInfo.

```
from fruits.models import FruitsInfo
FruitsInfo.objects.filter(price__gte=2)
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> FruitsInfo.objects.filter(price__gte=2)
<QuerySet [<FruitsInfo: USA apple>, <FruitsInfo: brazil pineapple>]>
>>>
```

```
from fruits.models import FruitsInfo
FruitsInfo.objects.filter(availability__lte=1000)
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> FruitsInfo.objects.filter(availability__lte=1000)
<QuerySet [<FruitsInfo: USA apple>]>
>>>
```

Atualização de registros

O Gerenciador de Objetos fornece um método chamado update() para atualizar um único registro ou vários registros. Quando uma atualização é realizada com sucesso, a saída será o número de registros afetados no banco de dados. O método update() é análogo à instrução UPDATE em SQL.

Os exemplos a seguir demonstram como o método update() pode ser utilizado.

Atualizando um único valor

A operação de atualização pode ser feita juntamente com um método filter() para especificar o registro que precisa ser atualizado. Vamos atualizar a origem das maçãs (id=1) na tabela FruitsInfo.

```
from fruits.models import FruitsInfo
record = FruitsInfo.objects.get(id=1)
print(record.origin)

FruitsInfo.objects.filter(id=1).update(origin='australia')

record = FruitsInfo.objects.get(id=1)
print(record.origin)
```

RESULTADO

```
>>> from fruits.models import FruitsInfo
>>> record = FruitsInfo.objects.get(id=1)
>>> print(record.origin)
USA
>>>
>>> FruitsInfo.objects.filter(id=1).update(origin='australia')
1
>>>
>>> record = FruitsInfo.objects.get(id=1)
>>> print(record.origin)
australia
>>>
```

No bloco de código acima, primeiro verificamos o valor original do campo `origin` do registro. Em seguida, atualizamos `origin` para "australia" e, finalmente, verificamos a operação de atualização chamando o campo `origin` usando o método `get()`. Como atualizamos apenas um único registro, a operação de atualização retorna o número de registros afetados, que é um.

Atualizando múltiplos valores

Nesta seção, veremos como atualizar vários campos em um registro. Para demonstrar isso, atualizaremos os campos de preço e disponibilidade no registro kiwi (id=4).

```
record = FruitsInfo.objects.get(id=4)
print(f"Price = {record.price}, Availability = {record.availability}")

FruitsInfo.objects.filter(id=4).update(price=0.55, availability=5000)

record = FruitsInfo.objects.get(id=4)
print(f"Price = {record.price}, Availability = {record.availability}")
```

RESULTADO

```
>>> record = FruitsInfo.objects.get(id=4)
>>> print(f"Price = {record.price}, Availability = {record.availability}")
Price = 1.00, Availability = 3000
>>>
>>> FruitsInfo.objects.filter(id=4).update(price=0.55, availability=5000)
1
>>> record = FruitsInfo.objects.get(id=4)
>>> print(f"Price = {record.price}, Availability = {record.availability}")
Price = 0.55, Availability = 5000
>>>
```

No bloco de código acima, definimos vários campos que precisam ser atualizados dentro da instrução de atualização. O resultado da operação de atualização ainda indica apenas um, porque atualizamos vários campos de um único registro.

Atualizando vários registros

Ao atualizar vários registros, a principal consideração é que o campo que está sendo atualizado deve estar presente em todos os registros. Se o campo não estiver disponível, isso resultará em um erro FieldDoesNotExist.

No próximo exemplo, atualizaremos o campo de disponibilidade de todos os registros para 10000. Conseguimos isso usando o método update() sem especificar um filtro.

```
FruitsInfo.objects.all().values()
FruitsInfo.objects.update(availability=10000)
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'australia', 'price': Decimal('2.50'), 'availability': 1000}, {'id': 2,
'name': 'orange', 'origin': 'canada', 'price': Decimal('1.25'), 'availability': 2000}, {'id': 3, 'name': 'pineapple', 'o
rigin': 'brazil', 'price': Decimal('3.00'), 'availability': 1500}, {'id': 4, 'name': 'kiwi', 'origin': 'china', 'price':
 Decimal('0.55'), 'availability': 5000}]>
>>>
>>> FruitsInfo.objects.update(availability=10000)
4
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'australia', 'price': Decimal('2.50'), 'availability': 10000}, {'id': 2,
 'name': 'orange', 'origin': 'canada', 'price': Decimal('1.25'), 'availability': 10000}, {'id': 3, 'name': 'pineapple',
'origin': 'brazil', 'price': Decimal('3.00'), 'availability': 10000}, {'id': 4, 'name': 'kiwi', 'origin': 'china', 'pric
e': Decimal('0.55'), 'availability': 10000}]>
>>>
```

O resultado da operação de atualização mostra que quatro registros foram afetados por essa atualização.

Excluindo um registro

O shell do Django fornece o método delete() para excluir registros de uma classe específica. Isso é análogo à instrução DELETE em SQL.

Excluindo um único registro

Ao excluir um único registro, devemos usar o método get(), pois ele retorna o objeto especificado diretamente. Vamos excluir o registro kiwi (id=4) da classe FruitsInfo.

```
FruitsInfo.objects.all().values()
FruitsInfo.objects.get(id=4).delete()
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'australia', 'price': Decimal('2.50'), 'availability': 10000}, {'id': 2,
 'name': 'orange', 'origin': 'canada', 'price': Decimal('1.25'), 'availability': 10000}, {'id': 3, 'name': 'pineapple',
'origin': 'brazil', 'price': Decimal('3.00'), 'availability': 10000}, {'id': 4, 'name': 'kiwi', 'origin': 'china', 'pric
e': Decimal('0.55'), 'availability': 10000}]>
>>>
>>> FruitsInfo.objects.get(id=4).delete()
(1, {'fruits.FruitsInfo': 1})
>>>
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'australia', 'price': Decimal('2.50'), 'availability': 10000}, {'id': 2,
 'name': 'orange', 'origin': 'canada', 'price': Decimal('1.25'), 'availability': 10000}, {'id': 3, 'name': 'pineapple',
'origin': 'brazil', 'price': Decimal('3.00'), 'availability': 10000}]>
>>>
```

O método delete() também retorna o número de registros afetados pela operação de exclusão.

Excluindo vários registros

O método `delete()` pode ser usado para excluir todos os registros de uma determinada classe, simplesmente especificando a operação de exclusão com o método `all()`. No exemplo a seguir, excluiremos todos os registros restantes da classe `FruitsInfo`.

```
FruitsInfo.objects.all().values()
FruitsInfo.objects.all().delete()
FruitsInfo.objects.all().values()
```

RESULTADO

```
>>> FruitsInfo.objects.all().values()
<QuerySet [{'id': 1, 'name': 'apple', 'origin': 'australia', 'price': Decimal('2.50'), 'availability': 10000}, {'id': 2,
 'name': 'orange', 'origin': 'canada', 'price': Decimal('1.25'), 'availability': 10000}, {'id': 3, 'name': 'pineapple',
'origin': 'brazil', 'price': Decimal('3.00'), 'availability': 10000}]>
>>>
>>> FruitsInfo.objects.all().delete()
(3, {'fruits.FruitsInfo': 3})
>>>
>>> FruitsInfo.objects.all().values()
<QuerySet []>
>>>
```

A partir da saída acima, podemos identificar que a operação de exclusão foi realizada em todos os três registros restantes.

Considerações finais

Neste artigo, aprendemos como usar o Django Shell para interagir com o banco de dados usando o Mapeador Objeto-Relacional (ORM) do Django. O Django Shell oferece todas as funcionalidades do shell padrão do Python, adicionando também as funcionalidades do framework Django diretamente ao ambiente do shell. Isso permite que os desenvolvedores testem e depurem interações com o banco de dados sem a necessidade de modificar o projeto Django.

Escrito em 24 de outubro de 2020